

BIDMATCH PROJECT PRESENTATION

BidMatch

AI 입찰공고 추천



기업 정보를 기반으로 적합한 입찰공고를 탐색하고 AI를 활용해 검색·요약·자격진단을 지원하는 서비스

ROLE

AI Backend · Search

Natural Language Search · LLM

2026.07 — 2026.08

TEAM PROJECT · 5 MEMBERS

BIDMATCH TEAM

TABLE OF CONTENTS

목차보기

전체 구성입니다.



1 프로젝트 개요

- 1.1 프로젝트 배경 및 목표
- 1.2 서비스 소개
- 1.3 나의 역할

2 시스템 구성

- 2.1 시스템 아키텍처
- 2.2 기술 스택

3 주요 기능 구현

- 3.1 자연어 기반 AI 검색
- 3.2 LLM 기반 공고 요약

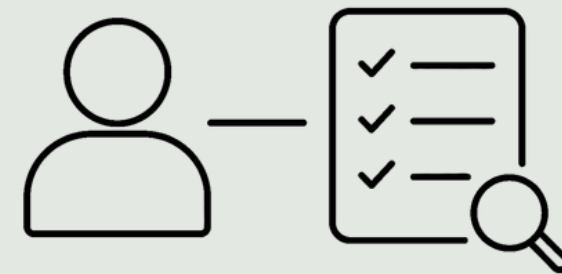
1.프로젝트 개요



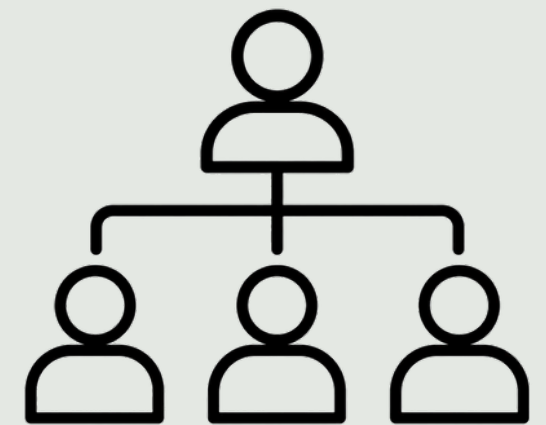
1. 추진 배경 및 목표



2. 서비스 소개



3. 주요 기능



4. 나의 역할

프로젝트 배경 및 목표



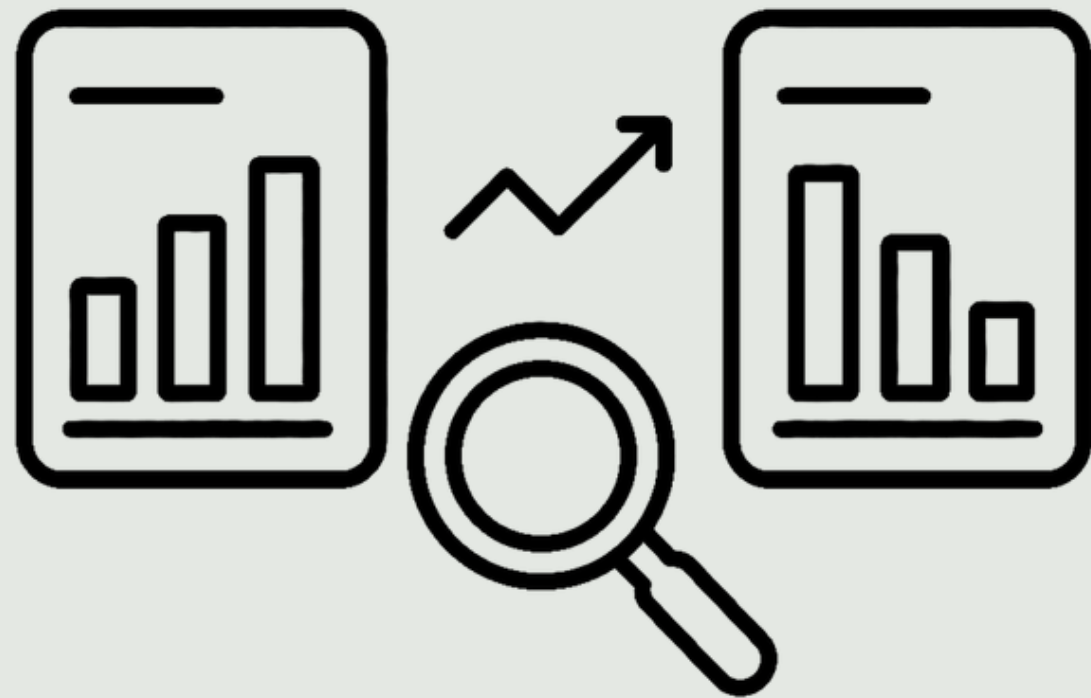
✔ 문제

반복적인 공고 탐색과 복잡한 조건 관리로 인해 적합한 입찰공고를 찾는 데 많은 리소스가 필요했습니다.

✔ 목표

기업정보와 관심 조건을 활용해 적합한 공고를 선별하고, AI를 통해 검색 및 입찰 참여 판단을 지원하는 서비스를 구현했습니다.

서비스 소개



✓ BidMatch

기업 정보를 기반으로 적합한 입찰공고를 탐색하고,
AI를 활용해 검색·요약·입찰 자격 판단을 지원하는 서비스

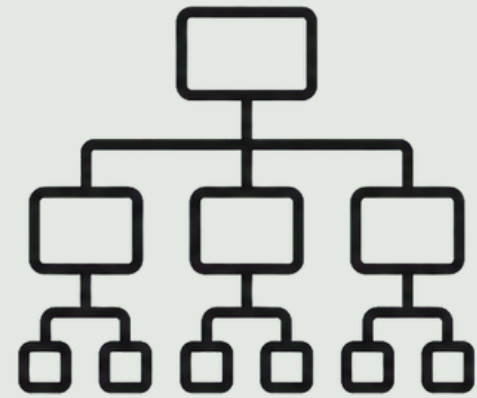
핵심 가치

- ✓ 기업 맞춤형 공고 선별
- ✓ 자연어 기반 공고 검색
- ✓ 근거가 보이는 입찰 자격 진단
- ✓ 나라장터 관련 업무 통합 지원

나의 역할

번호	구분	기능명	설명
1	검색	자연어 기반 AI 검색	자연어 질문에서 지역·키워드·예산 조건을 추출하고 의미 기반 검색 수행
2	공고	LLM 기반 공고 요약	긴 공고 내용을 핵심 내용 중심으로 요약
3	공고	AI 유사 공고 추천	현재 공고와 유사한 공고를 분석하여 관련 공고 추천
4	대시보드	대시보드	공고매칭,공고마감등 관련 알람제공 및 관심공고 일정을 한눈에 확인
5	공고	관심공고 관리	관심공고 저장·조회 및 마감 일정 관리
6	공고	엑셀 다운로드	조회한 공고 데이트럴 Excel 파일로 다운로드

2. 시스템 구성

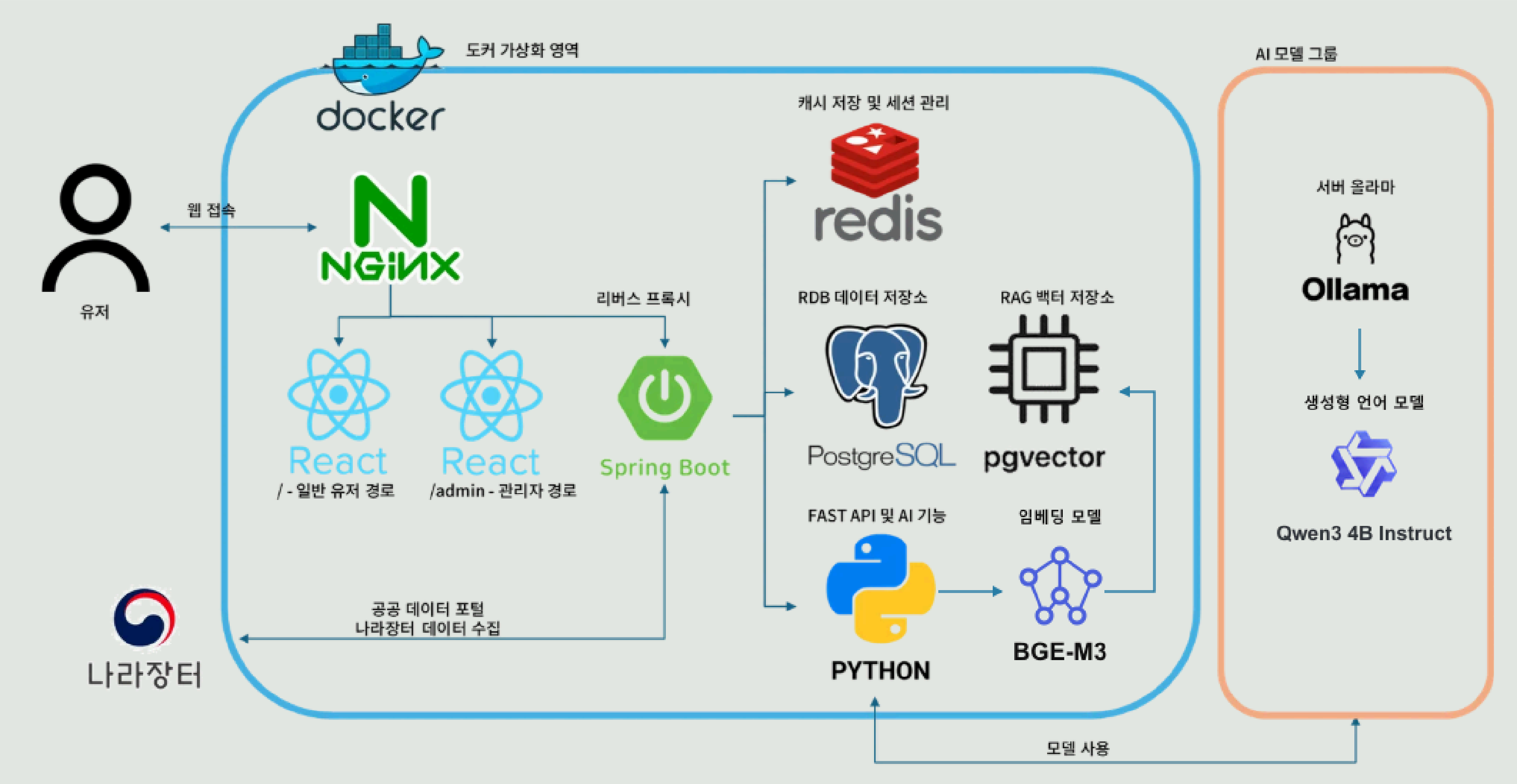


1.시스템 아키텍처



2.기술 스택

시스템 아키텍처



기술 스택

FRONTEND

화면 및 사용자 경험

React 18.3.1

컴포넌트 기반 사용자·관리자 화면 구성

Vite 5.4.11

개발 서버와 프론트엔드 빌드 환경

React Router DOM

화면 이동, 보호 경로 및 권한별 라우팅

Axios

Backend·AI API 요청과 응답 처리

Lucide React

일관된 메뉴·버튼 아이콘 UI 적용

SheetJS

조회 결과 및 관리 데이터 엑셀 출력

BACKEND

API 및 비즈니스 로직

Java 17

백엔드 서비스의 안정적인 실행 기반

Spring Boot 3.5.15

REST API와 애플리케이션 구성 통합

Spring Web

사용자·관리자 요청을 처리하는 API 제공

Spring Data JPA

회원·기업·광고 관계형 데이터 처리

Spring Security

로그인 인증과 사용자 권한 구분

Spring Validation

요청 데이터 형식과 필수값 검증

DATA & SUPPORT

데이터 및 운영 지원

PostgreSQL 16 · pgvector · JSONB

업무 데이터, 임베딩 벡터 및 비정형 AI 결과 저장

Redis 7.4 · Spring Session

로그인 세션, 인증번호, 임시 토큰 및 캐시 관리

Flyway · Mail · POI · Actuator

DB 변경, 이메일, 엑셀 처리와 서비스 상태 점검

기술 스택 – AI 및 배포 기술

AI API AI 서비스 기반

Python 3.12
AI 데이터 처리 및 모델 실행 환경

FastAPI · Uvicorn
검색·매칭·요약·챗봇·OCR API 제공

Pydantic Settings
요청 스키마 검증과 환경 설정 관리

HTTPX · Psycopg 3
외부 LLM 호출 및 PostgreSQL 직접 연동

AI MODEL 모델 및 검색 구조

BGE-M3
공고·기업 정보를 의미 벡터로 변환

BGE-Reranker-v2-M3
후보 공고의 관련도를 재평가해 순위 개선

Qwen 계열 LLM · Ollama
공고 요약·질의응답·Graph RAG 설명 생성

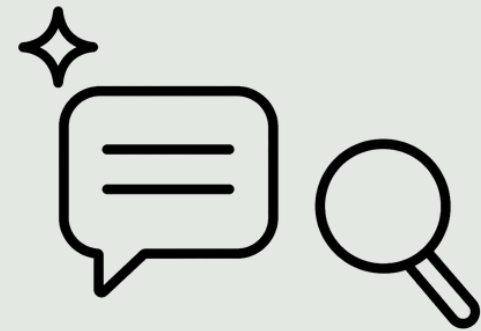
OCR · TEST · DEPLOY 인식, 품질 검증 및 운영

PaddleOCR · EasyOCR · OpenCV
사업자등록증 이미지 전처리, 문자 인식 및 정보 추출

Bruno · Pytest · JUnit 5
API 시나리오, AI·Backend 단위 및 통합 테스트

Docker · Nginx · Jenkins · CUDA
서비스 컨테이너화, 배포 자동화 및 GPU 모델 운영

3. 주요 기능 구현



1. 자연어 기반 AI 검색



2. 공고 3줄 요약

공고 검색

공고 목록

기업 사용자님

기업회원

→

로그아웃

유지보수

AI 모드

검색

초기화

옵션

검색조건

엑셀 다운로드

일치도	관심	공고명	기관명	예산(원)	게시일시	마감일시	상태
95%		2026년 하반기 보도안전시설물 유지보수공사 (단가계약) 제 R26BK01651913-000호	경기도 광명시	45,450,000	2026-07-27 17:25	2026-07-31 10:00 D-4	• 진행중
95%		2026년 제3차 대동지역 도로조명 유지보수 및 교체공사(단가계약) 제 R26BK01648829-000호	경기도 군포시	63,636,364	2026-07-27 14:42	2026-07-31 14:00 D-4	• 진행중
95%		전북대학교 AI JOB@JBNU 시스템 기능 추 가 및 유지보수 용역 제 R26BK01651475-000호	전북대학교	79,090,909	2026-07-27 14:28	2026-08-07 11:00 D-11	• 진행중
95%		교량시설물 유지보수 공사(탑동대교) 제 R26BK01647541-000호	경기도 오산시	285,027,273	2026-07-27 14:25	2026-08-04 10:00 D-8	• 진행중
		범용 전산해석 전후처리 소프트웨어 유지보수				2026-08-03 10:00	

주요 제공 정보

키워드와 기간·지역·예산 등의 조건을 조합해 원하는 입찰공고를 빠르게 탐색하도록 지원합니다.

- 검색어와 상세 조건을 기반으로 공고 목록 제공
- 자연어 검색에서 지역, 키워드, 예산 조건 추출
- AI 추천 점수와 추천 사유 확인
- 최신순, 마감임박순, 관련도순 정렬 제공
- 관심 공고 등록 및 관심공고함 연동
- 저장된 검색조건을 다시 불러와 반복 검색 지원
- 검색 결과 엑셀 다운로드 제공
- 공고명 클릭 시 상세 화면으로 이동
- 마감 공고와 진행 중 공고 상태 구분
- 검색 결과는 공고 탐색을 돕는 참고 정보로 제공

도입 배경 및 목적

사용자가 여러 검색 조건을 직접 조합하지 않아도, 자연어로 원하는 공고 후보를 빠르게 찾을 수 있도록 지원합니다.

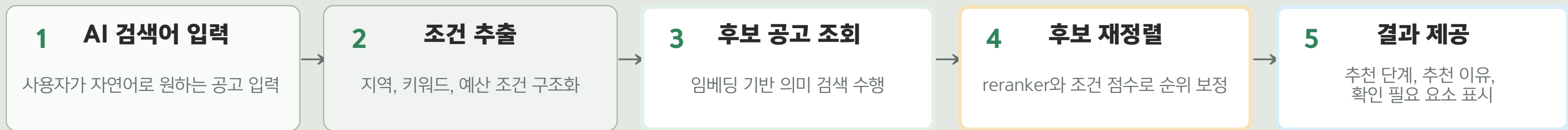
도입 이유

- 공고 수가 많아 사용자가 직접 조건을 조합하며 찾기 어려움
- 키워드만으로는 지역, 예산, 업종, 업무 의도를 함께 반영하기 어려움
- 오타, 유사 표현, 포괄적인 질문에서는 일반 검색 결과 품질이 낮아질 수 있음
- 검색 결과가 왜 추천되었는지 판단 근거를 함께 제공할 필요가 있음

도입 목적

- 자연어 질문에서 지역, 키워드, 예산 조건을 추출
- 임베딩 검색과 reranker를 이용해 의미가 가까운 공고 후보 조회
- 조건 점수와 추천 근거를 함께 반영해 검토 우선순위 제공
- 추천 이유와 확인 필요 요소를 표시해 사용자의 공고 검토 시간을 단축

기능 개요



사용자는 여러 조건을 직접 설정하지 않아도 자연어 질문만으로 검토할 공고 후보를 빠르게 확인할 수 있습니다.

사용자UI



사용자 UI 구성

- 1. **AI 검색 입력**
 - AI 검색 버튼을 누르면 지역, 키워드, 예산 조건을 추출해 검색합니다.
- 2. **예시 검색어 제공**
 - 예시를 클릭하면 검색어가 자동 입력되어 빠르게 AI 검색을 실행할 수 있습니다.
- 3. **검색 옵션 조정**
 - 조건 반영 버튼을 눌렀을 때만 검색에 적용되어 불필요한 재검색을 줄입니다.

사용 모델 및 선정 근거

구분	모델·기술	적용 역할	선정 이유
의미 검색	BGE-M3	검색어와 공고 내용을 임베딩 벡터로 변환	한국어 공고 문장과 자연어 검색어의 의미 유사도 검색에 적합
벡터 검색	pgvector	공고 임베딩 저장 및 유사 공고 후보 조회	DB 안에서 벡터 유사도 검색을 수행할 수 있어 백엔드 연동이 안정적
검색 재정렬	BGE Reranker v2 M3	1차 검색 후보를 다시 평가해 순위 보정	단순 벡터 검색에서 발생하는 부정확한 후보 노출을 줄임
조건 추출	규칙 기반 Parser	지역, 키워드, 예산 조건 추출	LLM 없이도 빠르고 예측 가능한 조건 추출 가능
오타 보정	자모 분해 + SequenceMatcher	지역명 오타 보정	수원 → 수원처럼 자연어 검색 입력 오류를 보완
최종 점수화	하이브리드 점수 로직	의미 점수, 재정렬 점수, 조건 점수 통합	검색 결과를 단순 유사도가 아닌 검토 우선순위로 정리

검색 데이터 셋 (1/2)

공고 원천 데이터와 검색용 임베딩 데이터

자연어 검색 품질을 높이기 위해 공고 기본 정보, 검색 문서, 임베딩 벡터, 검증 질문을 분리해 구성했습니다.

공고 기본 데이터

공고명, 기관, 예산, 마감일, 계약방법

검색 문서 데이터

과업 내용, 참가 자격, 면허, 공고 본문

임베딩 데이터

BGE-M3 기반 공고 의미 벡터

검색 평가 질문

지역, 예산, 오타, 복합 조건 검증

활용 흐름



검색 조건 유형을 주요 검증 영역으로 분류

- 지역 조건: 서울, 수원, 광주 등

예산 조건: 1억 이하, 2억 5천만 원 이하 등

업무 키워드: 시스템, 구축, CCTV, 네트워크 등
- 공고 분류: 공사, 용역, 물품 등

자격/면허: 필수 면허, 참가 자격

데이터 품질: 지역/예산/문서 누락 여부

※ 검색 문서와 임베딩 데이터는 자연어 검색 후보 추출 및 점수 계산에 사용됩니다.

검색 데이터 셋 (2/2)

검색 평가 데이터와 응답 구조

반복 가능한 검색 품질 확인을 위해 자연어 질문, 예상 추출 조건, 기대 결과 기준을 함께 관리했습니다.

검색 평가 질문 구성

- 지역 검색: 수원 시스템 구축 찾아줘
- 오타 검색: 수윙 시스템 구축 찾아줘
- 예산 검색: 2억 5천만 원 이하 사업 찾아줘
- 복합 검색: CCTV와 네트워크 구축이 함께 포함된 사업 찾아줘
- 포괄 검색: 광주 공고를 모아서 보여줘

검색 시나리오별 평가

- 일반 사용자: 자연어 검색어 입력 후 추천 후보 확인
- 기업 사용자: 예산/지역/업무 조건 기반 공고 후보 확인
- 검증 기준: 조건 추출, 후보 적합성, 추천 이유, 확인 필요 요소

검색 평가 레코드 예시

```
{
  "query": "수원 시스템 구축 찾아줘",
  "expectedConditions": {
    "regions": ["수원"],
    "keywords": ["시스템", "구축"],
    "maxBudget": null
  },
  "expectedResult": {
    "hasRegionMatch": true,
    "hasKeywordMatch": true,
    "showsReasons": true,
    "showsRiskFactors": true
  }
}
```

검색 질문과 기대 조건을 분리해 관리하여,
개선 전후 검색 결과를 같은 기준으로 비교할 수 있도록 구성했습니다.

검색 전처리

사용자의 자연어 검색어를 검색 가능한 조건과 임베딩 입력으로 변환합니다.



전처리 결과

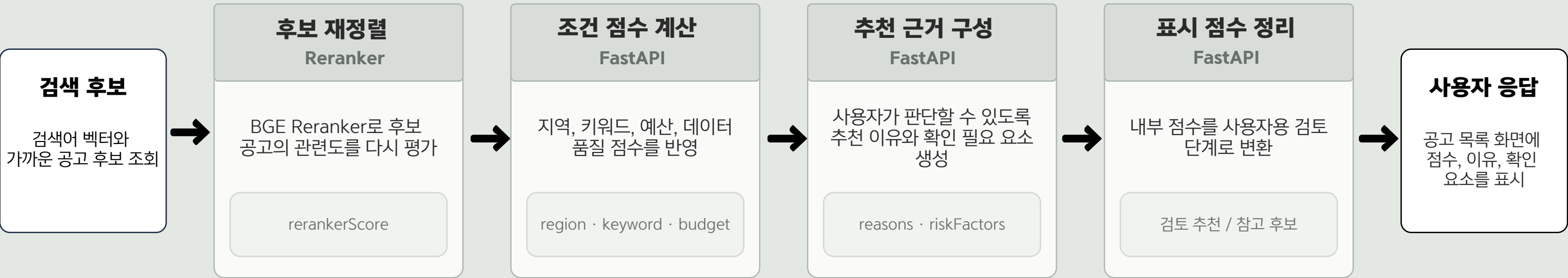
자연어 문장을 검색 조건과 임베딩 입력으로 변환

사용자 문장에서 지역, 키워드, 예산 조건을 추출하고, 오타를 보정해 벡터 검색과 조건 점수 계산에 사용할 수 있는 검색 요청 데이터로 구성했습니다.

원문 입력 → 검색어 정리 → 조건 추출 → 오타 보정 → 요청 구조화 → 임베딩 입력

검색 후처리

검색 후보를 의미 유사도와 조건 점수 기준으로 재정렬하고, 화면에 표시할 추천 결과로 정리합니다.



후처리 결과

검색 후보를 검토 우선순위 기준으로 정리

의미 유사도, 재정렬 점수, 지역·키워드·예산 조건을 조합해 후보 공고를 정렬하고, 사용자가 결과를 판단할 수 있도록 추천 이유와 확인 필요 요소를 함께 제공했습니다.

후보 조회 → reranker 재평가 → 조건 점수 계산 → 추천 근거 구성 → 표시 점수 정리 → 결과 표시

트러블 슈팅 (1/4)

번호	문제	원인	최종 대응
1	자연어가 단순 키워드처럼 처리됨	“수원 시스템 구축” 같은 문장에서 지역, 업무, 예산 조건이 분리되지 않음	지역 / 키워드 / 예산 조건을 구조화해 검색 조건으로 반영
2	지역 오타 입력 시 조건 누락	수령처럼 입력하면 지역명으로 인식하지 못함	자모 분해 + SequenceMatcher 기반 지역명 보정 적용
3	의미 유사도만으로 부정확한 후보 노출	벡터 유사도만 보면 실제 조건과 약한 공고도 상위 노출 가능	semanticScore + rerankerScore + 조건 점수 조합
4	조건 반영 후에도 순위가 불안정함	지역/예산/키워드 조건보다 의미 점수가 과하게 영향	regionScore, budgetScore, keywordScore 가중치 반영
5	결과 판단 근거가 부족함	점수만으로는 왜 추천되었는지 설명하기 어려움	추천 이유, 확인 필요 요소, 매칭 상세 정보를 함께 표시
6	점수가 정답률처럼 보임	사용자가 AI 점수를 “일치율/정답률”로 오해할 수 있음	UI 문구를 검토 추천 단계로 변경하고 내부 점수는 보조 정보로 표시
7	AI API 호출 경로 혼선	프론트가 FastAPI를 직접 호출하거나 운영 경로와 달라 오류 발생	front → Spring Backend → FastAPI proxy 구조로 정리
8	AI 응답 실패 시 화면 공백 가능	FastAPI 지연, 모델 로딩, 서버 오류 발생 가능	실패 안내 메시지 표시 및 일반 검색 유도 fallback 적용

트러블 슈팅 (2/4)

- 검색 품질 이슈

```
"query": "수원 시스템 구축 찾아줘",
"topK": 3,
"sort": "relevance"

"query": "수원 시스템 구축 찾아줘",
"extractedConditions": {
  "regions": [
    "수원"
  ],
  "keywords": [
    "시스템",
    "구축"
  ],
  "minBudget": null,
  "maxBudget": null
},
"results": [
  {
    "noticeId": "R26BK01638976",
    "title": "정자어린이공원 일원 노후관 교체공사 폐기물처리용역",
    "organization": "경기도 수원시",
    "region": null,
    "budgetAmount": 56720000,
    "postedAt": "2026-07-20T14:10:06+09:00",
    "deadline": "2026-07-24T10:00:00+09:00",
    "score": 39.41,
    "matchBreakdown": {
      "semanticScore": 47.05,
      "rerankerScore": 0.0,
      "finalScore": 39.41,
      "regionScore": 100.0,
      "keywordScore": 0.0,
      "budgetScore": null,
      "dataQualityScore": 100.0,
      "regionMatched": true,
      "matchedRegions": [
        "수원"
      ],
      "keywordMatches": [],
      "requiredLicenses": [

```



```
"query": "수원 시스템 구축 찾아줘",
"extractedConditions": {
  "regions": [
    "수원"
  ],
  "keywords": [
    "시스템",
    "구축"
  ],
  "minBudget": null,
  "maxBudget": null
},
"results": [
  {
    "noticeId": "R26BK01648480",
    "title": "국립박물관단지 스마트 수장고 시스템 구축 전락 수립",
    "organization": "행정중심복합도시 국립박물관단지 통합운영지원센터",
    "region": null,
    "budgetAmount": 40000000,
    "postedAt": "2026-07-24T13:19:12+09:00",
    "deadline": "2026-08-04T10:00:00+09:00",
    "score": 60.0,
    "matchBreakdown": {
      "semanticScore": 49.15,
      "rerankerScore": 1.29,
      "finalScore": 60.0,
      "regionScore": 20.0,
      "keywordScore": 100.0,
      "budgetScore": null,
      "dataQualityScore": 65.0,
      "regionMatched": false,
      "matchedRegions": [],
      "keywordMatches": [
        "시스템",
        "구축"
      ],

```

대응 방식

1. 자연어가 단순 키워드 처럼 처리되는 문제

- 사용자 검색어에서 불필요한 표현을 제거하고, 지역·키워드·예산 조건으로 구조화했습니다.
- 지역명 오타는 자모 분해와 SequenceMatcher 기반 유사도 비교로 보정했습니다.
- 예: 수원 시스템 구축 찾아줘 → 수원 / 시스템 / 구축
- CCTV, IT 같은 영문·축약어는 검색 키워드로 유지해 공고 후보 조회에 반영했습니다.

2. 의미 유사도만으로 부정확한 후보가 노출되는 문제 대응

- BGE-M3 임베딩으로 1차 후보를 찾고, BGE Reranker로 후보 공고의 관련도를 다시 평가했습니다.
- 이후 지역, 키워드, 예산, 데이터 품질 점수를 함께 반영해 최종 검토 우선순위를 계산했습니다.
- 점수는 정답률이 아니라 사용자가 먼저 확인할 공고를 정하는 “검토 추천 단계”로 표시했습니다.

트리플 슈팅 (3/4)

대응 방식

- Reranker 기반 검색 순위 보정

```
# reranker는 query와 passage 쌍을 입력으로 받음 - 0722 모하영
def _attach_reranker_scores(self, query: str, rows: list[dict[str, Any]]) -> None:
    """후보 공고에 rerankerScore를 붙여 최종 재정렬에 사용할 수 있게 합니다."""
    passages = [self._reranker_passage(row) for row in rows]
    scores = self._reranker.score(query, passages)
    for row, score in zip(rows, scores):
        row["reranker_score"] = score
```

1. Reranker 기반 재정렬
- 1차 벡터 검색으로 후보 공고를 조회했습니다
 - Query와 후보 공고를 다시 비교하여 Reranker Score를 계산했습니다.
 - 계산된 점수를 최종 검색 순위에 반영하여 의미적으로 더 적합한 공고가 상위에 노출되도록 개선했습니다.

트리플 슈팅 (4/4)

- Reranker 기반 검색 순위 보정

```
def _hybrid_score(
    self,
    semantic_score: float,
    reranker_score: float | None,
    region_score: float | None,
    keyword_score: float | None,
    budget_score: float | None,
    data_quality_score: float,
    graph_score: float | None,
) -> float:
    """의미 유사도와 조건별 점수를 활성 조건 기준 가중 평균으로 결합합니다."""
    components: list[tuple[float, float]] = [
        (data_quality_score, self._DATA_QUALITY_WEIGHT),
    ]

    if reranker_score is not None:
        components.append((reranker_score, self._RERANKER_WEIGHT))
        components.append((semantic_score, self._SEMANTIC_WEIGHT))
    else:
        components.append((semantic_score, 0.60))
    if region_score is not None:
        components.append((region_score, self._REGION_WEIGHT))
    if keyword_score is not None:
        components.append((keyword_score, self._KEYWORD_WEIGHT))
    if budget_score is not None:
        components.append((budget_score, self._BUDGET_WEIGHT))
    if graph_score is not None:
        components.append((graph_score, self._GRAPH_WEIGHT))

    weight_sum = sum(weight for _, weight in components)
    base_score = sum(score * weight for score, weight in components) / weight_sum

    return self._apply_intent_adjustment(
        base_score=base_score,
        semantic_score=semantic_score,
        reranker_score=reranker_score,
        region_score=region_score,
        keyword_score=keyword_score,
        budget_score=budget_score,
        graph_score=graph_score,
    )
```

대응 방식

2. Reranker 적용

- 의미 유사도만으로 후보를 정렬하던 방식을 보완했습니다.
- 1차 후보 공고를 검색어와 공고 본문 쌍으로 구성해 Reranker가 관련성을 다시 평가하도록 했습니다.
- 계산된 rerankerScore를 지역, 키워드, 예산, 데이터 품질 점수와 함께 최종 점수에 반영했습니다.

공고 요약 (+유사공고 추천)

🔍 AI 입찰 핵심 분석

- 사업 목적/범위: 부평구 공용차량 보험 서비스 제공용역 수행

- 참여 판단 핵심: 손해보험업 면허 소지 및 마감일 2026-08-13 06:00

- 원문 확인 필요: 계약방법 및 용역 범위의 구체적 내용

핵심 정보

배정 예산: 24,336,760원

마감 일시: 2026-08-13 06:00

수요기관: 인천시 부평구 시설관리공단

계약 방법: 수의계약

요구 면허: 손해보험업(자동차보험)/3828

⚠️ 확인 필요 요소

추천 57%

1순위

부흥로터리 노상공영주차장 바닥제어 무인주차시스템 설치위탁 운영 용역

AI 추천 · R26BK01618922

기관: 인천시 부평구 시설관리공단

예산: ₩ 0원

마감: 미정

검색어와 공고 내용이 일정 수준 이상 연결됩니다.

▼ AI 추천 근거 4개 보기

- 공고가 요청 지역과 연결됩니다.: 인천

- 공고 내용이 검색 업무 키워드 일부와 연결됩니다.: 부평구, 시설관리공단

- 기관 관계: 인천시 부평구 시설관리공단 (부평

추천 55%

2순위

2026년 부평구 동부권 재난대비 긴급 도로유지보수공사(연간단가 2차)

AI 추천 · R26BK01667314

기관: 인천광역시 종합건설본부

예산: 미정

마감: 2026년 08월 11일 10시 00분

검색어와 공고 내용이 일정 수준 이상 연결됩니다.

추천 54%

3순위

2026년 부평구 동부권 재난대비 긴급 도로유지보수공사 폐기물처리용역(연간...

AI 추천 · R26BK01667356

기관: 인천광역시 종합건설본부

예산: ₩ 21,439,000원

마감: 2026년 08월 11일 10시 00분

검색어와 공고 내용이 일정 수준 이상 연결됩니다.

추천 48%

2026년 공용차량

AI 추천 · R26BK016

기관: 부산교통공사

예산: ₩ 27,477,250

마감: 2026년 08월 1

검색어와 공고 내용이

주요 제공 정보

공고의 핵심 내용을 AI로 요약하고 유사 공고를 함께 제공합니다.

- AI 기반 공고 핵심 내용 3줄 요약
- 공고의 주요 정보 및 일정 요약
- 현재 공고와 유사한 입찰공고 추천
- 유사 공고의 주요 정보 비교 제공
- 관련 공고 추가 탐색 지원

24

도입 배경 및 목적

긴 입찰 공고를 직접 모두 읽기 전에, 참여 판단에 필요한 핵심 정보를 빠르게 파악하도록 지원합니다.

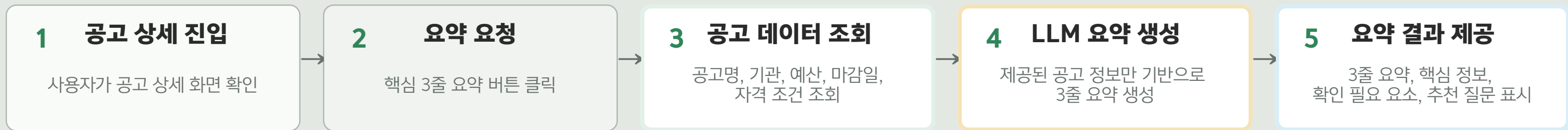
도입 이유

- 공고 상세 정보가 길어 사용자가 핵심 조건을 빠르게 파악하기 어려움
- 예산, 마감일, 계약방법, 자격 조건이 여러 영역에 흩어져 있음
- 원문 확인 전 1차 검토 시간을 줄일 보조 기능이 필요함

도입 목적

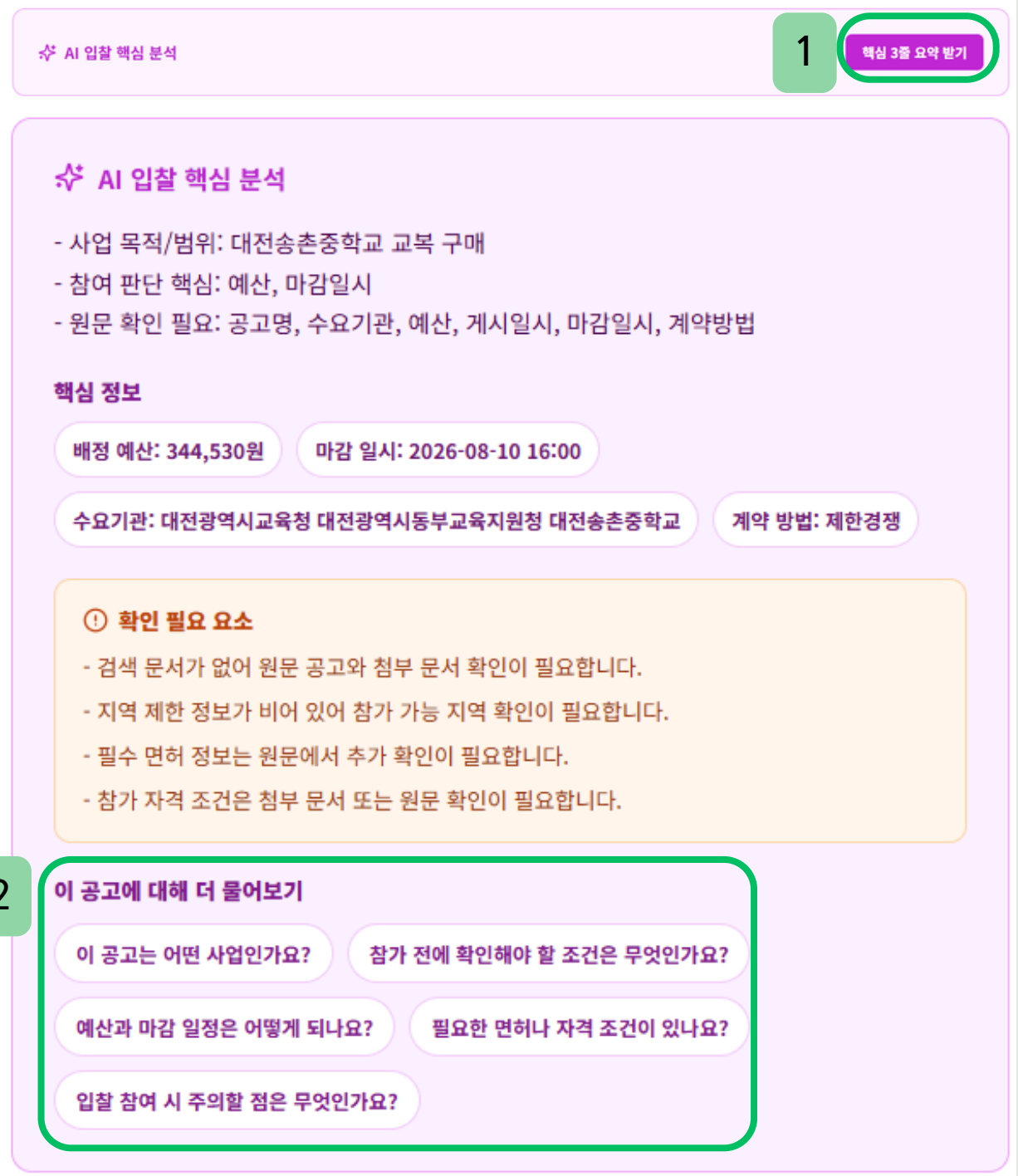
- 공고 상세 데이터를 기반으로 핵심 내용을 3줄로 요약
- 예산, 마감일, 수요기관, 계약방법 등 주요 정보를 별도 표시
- 면허, 자격, 원문 확인 필요 요소를 함께 안내
- 사용자가 바로 확인할 수 있는 추천 질문 제공

기능 개요



사용자는 원문을 모두 읽기 전에 공고의 핵심 조건과 확인 필요 사항을 빠르게 파악할 수 있습니다.

사용자 UI



사용자 UI 구성

1. **핵심 3줄 요약 받기**
 - 공고명, 수요기관, 예산, 마감일 등 상세 데이터를 기반으로 AI 요약을 요청합니다.
2. **추가 질문 확인**
 - 질문 클릭시 해당 공고 기준의 간단한 답변을 바로 확인할 수 있습니다

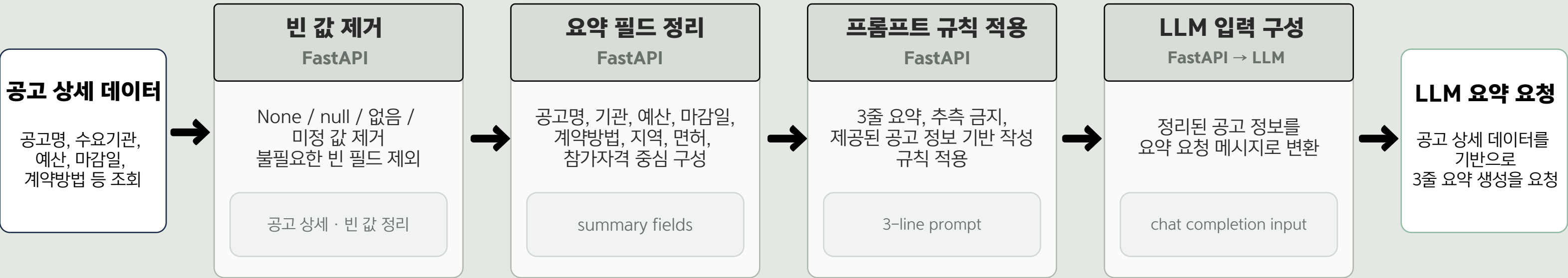
사용 모델 및 선정 근거

구분	모델·기술	적용 역할	선정 이유
주 요약 모델	Qwen3-4B	서버 적용 예정 3줄 요약 생성	한국어 요약 품질과 서버 자원 효율성이 우수하여 실제 서비스 적용 후보로 선정
로컬 검증 모델	qwen2.5:1.5b	로컬 기능 흐름 검증	개인 개발 PC에서도 실행 가능해 요약 API, 프론트 연동, fallback 흐름 검증에 사용
호출 방식	OpenAI 호환 Chat Completions3.5 Flash	LLM API 호출 표준화	모델이 바뀌어도 요청/응답 구조를 유지할 수 있어 모델 교체가 쉬움
실행 환경	Qwen3 4B Instruct	주 모델 장애 시 대체·로컬 테스트	로컬 실행이 가능하고 외부 API 장애 대응
응답 안정화	Prompt Builder + Response Formatter	3줄 제한, 빈 값 제거, 중복 제거, fallback 처리	LLM 응답이 길어지거나 None/null 값이 노출되는 문제를 줄이기 위해 적용

운영 검증 결과	기능 검증 결과 - qwen2.5:1.5b로 로컬 요약 API 호출 성공 - 공고 상세 화면에서 3줄 요약, 핵심 정보, 확인 필요 요소 표시 확인 - LLM 실패 시 템플릿 fallback 요약 반환 구조 확인 안정성 검증 - 서버 배포 시 Qwen3-4B 적용 예정 - OpenAI 호환 API 구조로 모델 교체 가능 - Temperature 0.2, Max Tokens 700 기준으로 요약 품질 조정 예정
----------	--

요약 전처리

LLM에 전달하기 전 공고 상세 데이터를 요약 가능한 입력 형태로 정리합니다.



전처리 결과

공고 상세 데이터를 LLM 요약에 적합한 입력 형태로 구성

빈 값과 불필요한 정보를 제거하고, 공고명, 예산, 마감일, 계약방법, 자격 조건 등 요약에 필요한 핵심 정보만 프롬프트로 구성했습니다.

공고 상세 조회 → 빈 값 제거 → 요약 필드 선별 → 프롬프트 규칙 적용 → LLM 입력 구성

요약 후처리

LLM이 생성한 응답을 상세 화면에 안정적으로 표시할 수 있는 결과 형태로 정리합니다.



후처리 결과

LLM 응답을 화면 표시용 3줄 요약으로 정규화

3줄 초과 응답, 중복 문장, 빈 값 표현을 제거하고 요약, 핵심 정보, 확인 필요 요소, 추천 질문으로 분리해 상세 화면에 안정적으로 표시했습니다.

LLM 응답 → 응답 정규화 → 3줄 제한 → 빈 값·중복 제거 → fallback 처리 → 요약 결과 표시

트러블 슈팅

번호	문제	원인	최종 대응
1	줄 수가 불안정함	LLM 응답이 자유 문장 형태라 3줄 기준을 지키지 않을 수 있음	3줄 요약 형식으로 정규화
2	빈 값/중복 문장 노출 가능	DB에 없는 값이 None/null/없음으로 들어오거나 같은 정보가 반복 출력됨	빈 값 필터링 및 중복 제거
3	LLM 오류 시 화면 깨짐 가능	로컬 모델/API 호출 실패, timeout, 모델 미응답 가능성 존재	template_fallback 기본 요약 제공
4	제공 정보 외 추측 가능	LLM이 공고에 없는 내용을 일반 지식으로 보완하려는 경향이 있음	공고 DB 정보 안에서만 요약 제한
5	후속 확인 동선 부족	요약만 제공하면 사용자가 다음에 무엇을 확인해야 할지 알기 어려움	추천 질문과 즉시 답변 제공

트러블 슈팅 (1/2)

1.요약 형식 표준화

```
async def summarize(self, notice_id: str) -> NoticeSummaryResponse:
    """공고 ID로 상세 데이터를 조회하고 LLM 요약 또는 템플릿 요약을 반환합니다."""
    notice = self._repository.fetch_notice(notice_id)
    if notice is None:
        raise ValueError("공고를 찾을 수 없습니다.")

    try:
        summary = self._normalize_summary(await self._generate_llm_summary(notice))
        generation_mode = "llm"
    except Exception:
        summary = self._build_template_summary(notice)
        generation_mode = "template_fallback"
```

2. 빈 값·추측 문장 제거

```
def _normalize_summary(self, summary: str) -> str:
    """LLM 응답에서 제목성 문장을 제거하고 상세 화면용 3줄 요약으로 정리합니다."""
    normalized_lines: list[str] = []
    seen_lines: set[str] = set()
    for raw_line in summary.splitlines():
        line = self._clean_summary_line(raw_line)
        if not line:
            continue
        dedupe_key = self._summary_dedupe_key(line)
        if dedupe_key in seen_lines:
            continue

        seen_lines.add(dedupe_key)
        normalized_lines.append(f"- {line}")
        if len(normalized_lines) == self._SUMMARY_LINE_LIMIT:
            break
    if not normalized_lines:
        raise ValueError("LLM 요약 응답에 표시할 문장이 없습니다.")
    return "\n".join(normalized_lines)
```

대응 방식

1.요약 형식 표준화

- LLM 응답을 후처리해 상세 화면용 3줄 요약으로 정리했습니다.
- 빈 줄, 제목성 문장, 중복 문장을 제거해 읽기 쉬운 형태로 변환했습니다.

2. 빈 값·추측 문장 제거

- None, null, 없음, 미정 같은 빈 값 표현이 화면에 노출되지 않도록 필터링했습니다.
- 공고 DB에 있는 제목, 기관, 예산, 마감일 등 확인 가능한 정보 중심으로 요약을 제한했습니다.

트러블 슈팅 (2/2)

3.fallback 요약 제공

```
def _build_template_summary(self, notice: dict[str, Any]) -> str:
    """LLM 호출 실패 시 화면이 깨지지 않도록 기본 요약 문장을 생성합니다."""
    title = notice.get("bid_ntce_nm") or "해당 공고"
    organization = notice.get("dminstt_nm") or notice.get("ntce_instt_nm") or
    "수요기관 미상"
    budget = self._format_money(notice.get("budget_amount"))
    deadline = self._format_datetime(notice.get("bid_clse_dt"))
    contract_method = notice.get("cntrct_cncls_mthd_nm")
    return "\n".join([
        f"- 사업 목적/범위: {title}는 {organization}에서 발주한 공고입니다.",
        self._fallback_schedule_line(budget, deadline, contract_method),
        "- 상세 참가 자격과 과업 조건은 원문 또는 첨부 문서 확인이 필요합니다.",
    ])
```

대응 방식

3.fallback 요약 제공

- LLM 호출 실패 시 try-except로 예외를 처리했습니다.
- 실패 상황에서도 template_fallback 요약을 제공해 화면 공백이나 오류 노출을 방지했습니다.



MARKETING PROJECT PRESENTATION

경청해주셔서 감사합니다!

본 프로젝트 제안서에 대해 문의할 점이 있으시면 언제든지 문의주세요.